

PROJECT PROPOSAL

Date of proposal:
Project Title: A RAG Based Multi Agent System for PC Build and Laptop Recommendations in Singapore
Group ID (As Enrolled in Canvas Class Groups): Group Members (name , Student ID):
Sponsor/Client: <i>(Company Name, Address and Contact Name, Email, if any)</i>
Background/Aims/Objectives: Choosing a computer requires users to compare configurations, local prices and user feedback while balancing budget and intended use. Desktop buyers also need to check component compatibility, while laptop buyers must consider configuration differences and portability. This project aims to develop a conversational recommendation system for users purchasing computer components or laptops in Singapore. It will interpret user requirements, retrieve relevant product information and reviews, recommend suitable options with traceable sources, and support follow-up adjustments. The objectives are to improve transparency in computer selection and evaluate whether specialised agents improve recommendation quality compared with a single-agent baseline.
Project Descriptions: Scope and user interaction. The proposed web application will accept requirements such as budget in SGD, intended software or games, preferred device type and existing components. It will ask for essential missing information, present desktop build lists or specific laptop configurations, explain trade-offs, and allow users to compare options or request changes while retaining selected constraints. Data foundation. Initial collection on 6 September 2026 produced 2,236 product-variant price records from Dynacore Singapore and Vii PC Singapore, together with 15 user-review excerpts obtained through public web retrieval. Records retain source links and collection timestamps. These review excerpts are a small sample; reviewer location and exact configuration matches are not always verified. Additional product specifications and review coverage are needed. RAG pipeline. Product records will be cleaned and kept separate by configuration. Long descriptions will be divided into meaningful passages, while short reviews will remain intact. Embeddings and source metadata will be stored in a vector database for retrieval. Current prices and stock will remain available through structured queries. Retrieved evidence will support explanations, with insufficient evidence identified explicitly.

Agent reasoning and planning. The proposed method is Plan, Execute, Review and Replan. A coordinating agent will maintain the user requirements and an explicit task plan. Desktop and laptop agents will propose candidates; an evidence agent will retrieve feedback; a review agent will assess constraint satisfaction and source support. Reasoning will be demonstrated through task decomposition, evidence-backed decisions, trade-offs and recorded plan changes.

Plan method. First, the coordinator will distinguish hard constraints, such as a maximum budget and locked components, from preferences such as low noise. Unknown requirements will remain explicit, and essential omissions will trigger a question. It will then decompose the request into candidate retrieval, configuration selection, validation, evidence retrieval and recommendation tasks. Each task will specify its ID, assigned agent, tool, required inputs, dependencies, expected output, completion criterion and status. For example, review retrieval requires candidate product IDs. After candidate selection, review retrieval and programmatic checks can run in parallel. Desktop and laptop planning will both run only when comparison is needed.

Execute and review. Agents will execute ready tasks against the same data snapshot and return structured results containing product and offer IDs, source references and missing-data flags. The reviewer will compare results with the original requirements and tool checks. A plan step is complete only when its stated output and checks are available. Conflicting evidence will be reported rather than resolved by agent voting. The user will see a concise decision record linking each recommendation to the relevant requirement, evidence and limitation.

Replan and stopping rules. A failed check, unavailable item or changed user requirement will trigger a revised plan. The coordinator will retain valid results and locked components, replace affected tasks and invalidate their dependent checks. Hard constraints will not be relaxed without user agreement. The proposed limit is two automatic revision rounds; the workflow then returns a feasible recommendation or explains unresolved constraints. Tool failures will be recorded separately from product infeasibility.

Illustrative planning scenario. For a desktop request with a S\$2,000 ceiling, 32GB RAM and a locked GPU, the plan will reserve the GPU cost, retrieve compatible candidates within the remaining budget, assemble a configuration, check its total and specifications, and retrieve feedback. If the total exceeds the ceiling, replanning will seek alternatives among unlocked components and repeat affected checks. If none are feasible, the system will explain the shortfall and ask which constraint the user wishes to change.

Backend and validation. FastAPI will expose the application services, and LangChain will support model and retrieval interactions. Programmatic tools will calculate totals, filter stock and check compatibility against available specifications. Missing specifications will produce an unknown status rather than a compatibility pass. The interface will show configuration details, prices, source dates and unresolved limitations.

Evaluation and deliverables. The project will deliver a data preparation and indexing pipeline, a RAG-based multi-agent backend and a web interface for recommendations and adjustments. Evaluation will compare single-agent and multi-agent approaches using the same data and test requirements, measuring constraint satisfaction, source accuracy, known compatibility-conflict detection, adjustment success, response time and model usage cost.